Modelación y Representación Formal de Problemas de Optimización Combinatoria y Satisfacción de Restricciones

Cervantes Araujo Carlos Iván / Manjarrez Olivas Mariana
Ingeniería en Sistemas Computacionales
Tópicos de Inteligencia Artificial
Instituto Tecnológico de Culiacán

6 de octubre de 2026

Resumen

En este trabajo se presenta un estudio analítico riguroso sobre la descripción y representación matemática de cuatro problemas emblemáticos de la optimización combinatoria y la satisfacción de restricciones: el Problema de Programación de Trabajos en Taller (JSSP), el Problema de las N-Reinas, el Árbol de Expansión Mínima (MST) y el Problema del Agente Viajero (TSP). Se abordan las restricciones inherentes a cada problema y se discuten las estructuras de codificación canónicas (permutaciones con y sin repetición, formulaciones matriciales y grafos disyuntivos), demostrando cómo una selección adecuada del espacio de representación minimiza la complejidad algorítmica y asegura la factibilidad de las soluciones exploradas.

1. Introducción

La optimización combinatoria y la satisfacción de restricciones constituyen dos de los pilares más determinantes de las ciencias de la computación contemporáneas, la investigación de operaciones y la inteligencia artificial. Múltiples actividades industriales y logísticas fundamentales —tales como la secuenciación de procesos de manufactura, la logística de última milla, el tendido de redes de infraestructura y el diseño de sistemas de decisión autónomos— dependen de la capacidad de evaluar y seleccionar configuraciones óptimas dentro de espacios de búsqueda discretos y de cardinalidad combinatoria astronómica.

Muchos de estos desafíos pertenecen a la clase de complejidad **NP-Hard** o se formulan como Problemas de Satisfacción de Restricciones (**CSP**), lo cual inhabilita el uso de técnicas de enumeración exhaustiva debido a la explosión combinatoria. Por consiguiente, la eficacia de cualquier método de búsqueda (ya sea exacto, heurístico o metaheurístico) depende directamente de dos factores: una **descripción rigurosa del problema** con sus restricciones operativas, y una **representación del estado** matemáticamente eficiente. Una codificación adecuada restringe el espacio de soluciones hacia regiones factibles, simplifica la definición de vecindades y optimiza el costo computacional de las funciones de aptitud.

El presente trabajo examina a profundidad la descripción detallada y las representaciones canónicas de cuatro problemas paradigmáticos: el **Problema de Programación de Trabajos en Taller (JSSP)**, el **Problema de las N-Reinas (N-Queens)**, el **Árbol de Expansión Mínima (MST)** y el **Problema del Agente Viajero (TSP)**.

2. Desarrollo de los Problemas

2.1. Problema de Programación de Trabajos en Taller (JSSP)

2.1.1. Descripción detallada del problema

El JSSP (*Job Shop Scheduling Problem*) es un problema de optimización combinatoria de clase NP-Hard enfocado en la asignación temporal de tareas en entornos productivos de manufactura discreta [3].

- **Conjunto de instancias:**
  - Un conjunto $J = \{J_1, J_2, \ldots, J_n\}$ de n trabajos independientes.
  - Un conjunto $M = \{M_1, M_2, \ldots, M_m\}$ de m máquinas heterogéneas.
- **Secuencia tecnológica:** Cada trabajo J_i consiste en una secuencia estrictamente ordenada de m operaciones consecutivas:

  $$O_{i1} \to O_{i2} \to \cdots \to O_{im}$$

  donde cada operación O_{ij} debe procesarse en una máquina designada $\mu(O_{ij}) \in M$ durante un tiempo de procesamiento ininterrumpido $p_{ij} \in \mathbb{R}^+$.
- **Restricciones fundamentales:**
  1. *Precedencia tecnológica:* Para cada trabajo J_i, la operación O_{ik} únicamente puede iniciar una vez concluida su predecesora directa $O_{i,k-1}$:

     $$S_{ik} \geq S_{i,k-1} + p_{i,k-1} \tag{1}$$

     donde S_{ik} denota el instante de inicio de la operación.
  2. *Capacidad unísona (Disyuntiva):* Cada máquina M_k puede procesar a lo sumo una operación a la vez. Si dos operaciones O_{ij} y O_{ab} compiten por la misma máquina ($\mu(O_{ij}) = \mu(O_{ab}) = M_k$):

     $$S_{ij} \geq S_{ab} + p_{ab} \quad \vee \quad S_{ab} \geq S_{ij} + p_{ij} \tag{2}$$

  3. *No apropiación (Non-preemption):* Las operaciones no pueden ser interrumpidas ni fragmentadas una vez iniciadas.
- **Función Objetivo:** Determinar los instantes de inicio $S_{ij} \geq 0$ de modo que se minimice el tiempo total de terminación de todos los trabajos, denominado *Makespan* ($C_{\text{máx}}$):

  $$\min C_{\text{máx}} = \max_{1 \leq i \leq n} (S_{im} + p_{im}) \tag{3}$$

2.1.2. Representación formal

1. **Permutación con Repetición (Vector de Trabajos):** Se define un vector unidimensional V de longitud $n \times m$. Cada identificador de trabajo $i \in \{1, \ldots, n\}$ figura exactamente m veces en el vector. La k-ésima aparición del índice i representa la k-ésima operación del trabajo J_i.

   Por ejemplo, para $n = 3$ y $m = 2$:

   $$V = [1,\ 2,\ 1,\ 3,\ 2,\ 3]$$

   Esta codificación garantiza que cualquier permutación válida produzca siempre una solución factible respecto a la precedencia tecnológica.

2. **Grafo Disyuntivo de Roy-Sussmann:** Se modela como un grafo dirigido $G = (V_G, C \cup D)$, donde V_G contiene todos los nodos de operaciones más dos ficticios (origen 0 y sumidero $*$). C agrupa los arcos conjuntivos dirigidos (precedencia interna) y D representa los arcos disyuntivos no dirigidos (conflicto de máquina). Resolver el problema equivale a orientar cada arista disyuntiva eliminando ciclos y minimizando la trayectoria del camino crítico.

2.2. Problema de las N Reinas (N-Queens)

2.2.1. Descripción detallada del problema

El problema de las N-Reinas es un clásico Problema de Satisfacción de Restricciones (CSP) derivado del acertijo propuesto por Max Bezzel en 1848 [4].

- **Definición formal:** Dado un tablero bidimensional homogéneo de tamaño $N \times N$, se requiere posicionar N reinas del juego de ajedrez de forma tal que ninguna reina ataque a otra en un solo movimiento.
- **Restricciones de no colisión:** Dadas dos reinas posicionadas en las coordenadas (r_i, c_i) y (r_j, c_j) con $i \neq j$:
  1. *Fila unívoca:* $r_i \neq r_j$
  2. *Columna unívoca:* $c_i \neq c_j$
  3. *Diagonal independiente:* $|r_i - r_j| \neq |c_i - c_j|$
- **Objetivo:** Determinar una configuración X que reduzca a cero la función de conflictos mutuos:

$$f(X) = \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} \text{conflicto}(q_i, q_j) = 0 \tag{4}$$

2.2.2. Representación formal

1. **Matriz Binaria:** Matriz $A \in \{0,1\}^{N \times N}$ donde $A_{ij} = 1$ si hay una reina en la fila i, columna j. Esta representación es ineficiente computacionalmente dado que induce un espacio de búsqueda desmedido de $\binom{N^2}{N}$ estados.
2. **Vector de Permutación Posicional (Canónica):** Asumiendo intrínsecamente que cada fila $i \in \{1, \ldots, N\}$ alberga a exactamente una reina, el estado se compacta en un vector $Q = [q_1, q_2, \ldots, q_N]$, donde $q_i \in \{1, \ldots, N\}$ representa la columna ocupada por la reina de la fila i.

   Si forzamos que Q sea una permutación sin elementos repetidos ($\forall i \neq j \implies q_i \neq q_j$), **las restricciones de fila y columna quedan satisfechas por definición**. La verificación de restricciones se reduce únicamente a la condición diagonal:

$$|i - j| \neq |q_i - q_j| \quad \forall i < j \tag{5}$$

2.3. Árbol de Expansión Mínima (MST)

2.3.1. Descripción detallada del problema

El MST (*Minimum Spanning Tree*) es un problema fundamental de optimización sobre grafos que pertenece a la clase de complejidad polinomial **P**, resoluble mediante algoritmos voraces (*greedy*) como Kruskal o Prim [2].

- **Definición formal:** Dado un grafo no dirigido y conexo $G = (V, E)$, donde V es el conjunto de n vértices ($|V| = n$), E es el conjunto de m aristas ($|E| = m$), y $w : E \to \mathbb{R}$ es una función de costo que asocia un peso $w(e)$ a cada arista $e \in E$.
- **Propiedades topológicas:** Se debe hallar un subconjunto de aristas $T \subseteq E$ tal que:
  1. Conecte a la totalidad de los vértices V.
  2. Sea acíclico (estructura de árbol libre), lo cual implica que $|T| = n - 1$.
- **Función Objetivo:** Minimizar el peso total acumulado de las aristas seleccionadas:

$$\min W(T) = \sum_{e \in T} w(e) \tag{6}$$

2.3.2. Representación formal

1. **Vector Binario de Incidencia de Aristas:** Considerando un orden fijo de las aristas $E = \{e_1, e_2, \ldots, e_m\}$, se construye un vector $X = [x_1, x_2, \ldots, x_m]$ donde:

$$x_k = \begin{cases} 1 & \text{si } e_k \in T \\ 0 & \text{en caso contrario} \end{cases} \tag{7}$$

   sujeto a $\sum_{k=1}^{m} x_k = n-1$ y a la verificación de aciclicidad mediante estructuras *Union-Find*.

2. **Código de Prüfer:** En grafos completos K_n, un árbol generador etiquetado puede codificarse unívocamente mediante una secuencia de longitud $n-2$ con valores en $\{1, \ldots, n\}$. Esta representación establece una biyección exacta que garantiza la ausencia total de ciclos sin requerir restricciones auxiliares.

2.4. Problema del Agente Viajero (TSP)

2.4.1. Descripción detallada del problema

El TSP (*Traveling Salesperson Problem*) es uno de los problemas de optimización combinatoria NP-Hard más estudiados en la teoría computacional [1].

- **Definición formal:** Dado un conjunto de n ciudades $V = \{1, 2, \ldots, n\}$ y una matriz de distancias $D = [d_{ij}]_{n \times n}$, donde $d_{ij} \geq 0$ cuantifica el costo o distancia de traslado desde la ciudad i hasta la ciudad j.
- **Condición de Ciclo Hamiltoniano:** El agente debe partir desde una ciudad inicial arbitraria, visitar cada una de las $n - 1$ ciudades restantes exactamente una sola vez y regresar al punto de partida.
- **Función Objetivo:** Encontrar una permutación π de los índices $\{1, \ldots, n\}$ que minimice la distancia euclidiana o métrica total del ciclo:

$$\min L(\pi) = \left(\sum_{i=1}^{n-1} d_{\pi(i),\pi(i+1)} \right) + d_{\pi(n),\pi(1)} \tag{8}$$

2.4.2. Representación formal

1. **Permutación Directa (Ruta Ordinal):** Representación estándar para metaheurísticas (Búsqueda Tabú, Algoritmos Genéticos, Recocido Simulado). Se modela como un vector $\pi = [\pi_1, \pi_2, \ldots, \pi_n]$ donde cada ciudad figura una sola vez. Por ejemplo, $\pi = [3, 1, 4, 2, 5]$ representa la trayectoria $3 \to 1 \to 4 \to 2 \to 5 \to 3$. Previene intrínsecamente visitas repetidas y omisiones.

2. **Formulación Matricial Binaria (MILP / Miller-Tucker-Zemlin):** Se define una matriz $X \in \{0,1\}^{n \times n}$ donde $x_{ij} = 1$ si el trayecto directo de i a j forma parte del recorrido. Sujeta a las restricciones de grado unitario:

$$\sum_{j=1, j \neq i}^{n} x_{ij} = 1 \quad \forall i \in V, \qquad \sum_{i=1, i \neq j}^{n} x_{ij} = 1 \quad \forall j \in V \tag{9}$$

y a las restricciones de eliminación de subciclos (*subtours*) de Miller-Tucker-Zemlin:

$$u_i - u_j + n x_{ij} \leq n - 1 \quad \forall i, j \in \{2, \ldots, n\},\ i \neq j \tag{10}$$

donde $u_i \in \mathbb{R}$ son variables auxiliares continuas.

3. Cuadro Comparativo

Problema	**Complejidad**	**Espacio de Búsqueda**	**Representación Canónica**	**Restricción Crítica**
JSSP	NP-Hard	Permutaciones disyuntivas	Vector con repetición / Grafo disyuntivo	Precedencia tecnológica y no solapamiento
N**-Reinas**	CSP (NP-C)	$N!$ (con restricción columna/fila)	Permutación posicional ($Q \in S_N$)	No colisión en diagonales absolutas
MST	P (Polinomial)	n^{n-2} árboles en K_n	Vector booleano de aristas / Código Prüfer	Conectividad global y aciclicidad ($n-1$ aristas)
TSP	NP-Hard	$\frac{(n-1)!}{2}$ (simétrico)	Permutación directa (π) / Matriz binaria MILP	Visita unívoca y supresión de subciclos

Cuadro 1: Comparativa estructural y computacional de los problemas modelados.

4. Conclusión

El análisis comparativo desarrollado evidencia que la modelación matemática de un problema no se reduce a la mera descripción de sus restricciones operativas, sino que exige una correspondencia directa con su codificación computacional.

En problemas de optimización combinatorial NP-Hard como el **JSSP** y el **TSP**, el empleo de representaciones basadas en permutaciones permite desacoplar la complejidad geométrica del espacio de búsqueda respecto a la factibilidad estructural, permitiendo que los operadores de vecindad operen eficientemente sobre el costo objetivo (*Makespan* o distancia acumulada). En contraparte, en problemas de satisfacción de restricciones como las N**-Reinas**, la codificación vectorial elimina por diseño dos de las tres familias de restricciones (filas y columnas), reduciendo la complejidad a la comprobación de diagonales algebraicas. Finalmente, en problemas tratables polinomialmente como el **MST**, la combinación de representaciones compactas con algoritmos voraces garantiza la convergencia al óptimo global sin recurrir a aproximaciones heurísticas.

En conclusión, la elección adecuada del modelo de datos y de la representación del estado fundamenta el éxito computacional necesario para resolver problemas combinatorios de gran escala en entornos industriales, computacionales y logísticos.

Referencias

[1] Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). *The Traveling Salesman Problem: A Computational Study*. Princeton University Press. https://doi.org/10.1515/9781400841103

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4ta ed.). MIT Press.

[3] Garey, M. R., Johnson, D. S., & Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2), 117–148. https://doi.org/10.1287/moor.1.2.117

[4] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4ta ed.). Pearson.

[5] Talbi, E.-G. (2009). *Metaheuristics: From Design to Implementation*. John Wiley & Sons. https://doi.org/10.1002/9780470496916